CHARUSAT®
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FINAL YEAR PROJECT • 2026

Path Guiding Robot

- An IoT-based autonomous campus navigation
- A System with smart campus analysis.

PRESENTED BY

Maulik Pithiya (23BSIT119)

Sujal Pandya (23BSIT055)

Abhishek Parmar (23BSIT060)

UNDER GUIDANCE OF

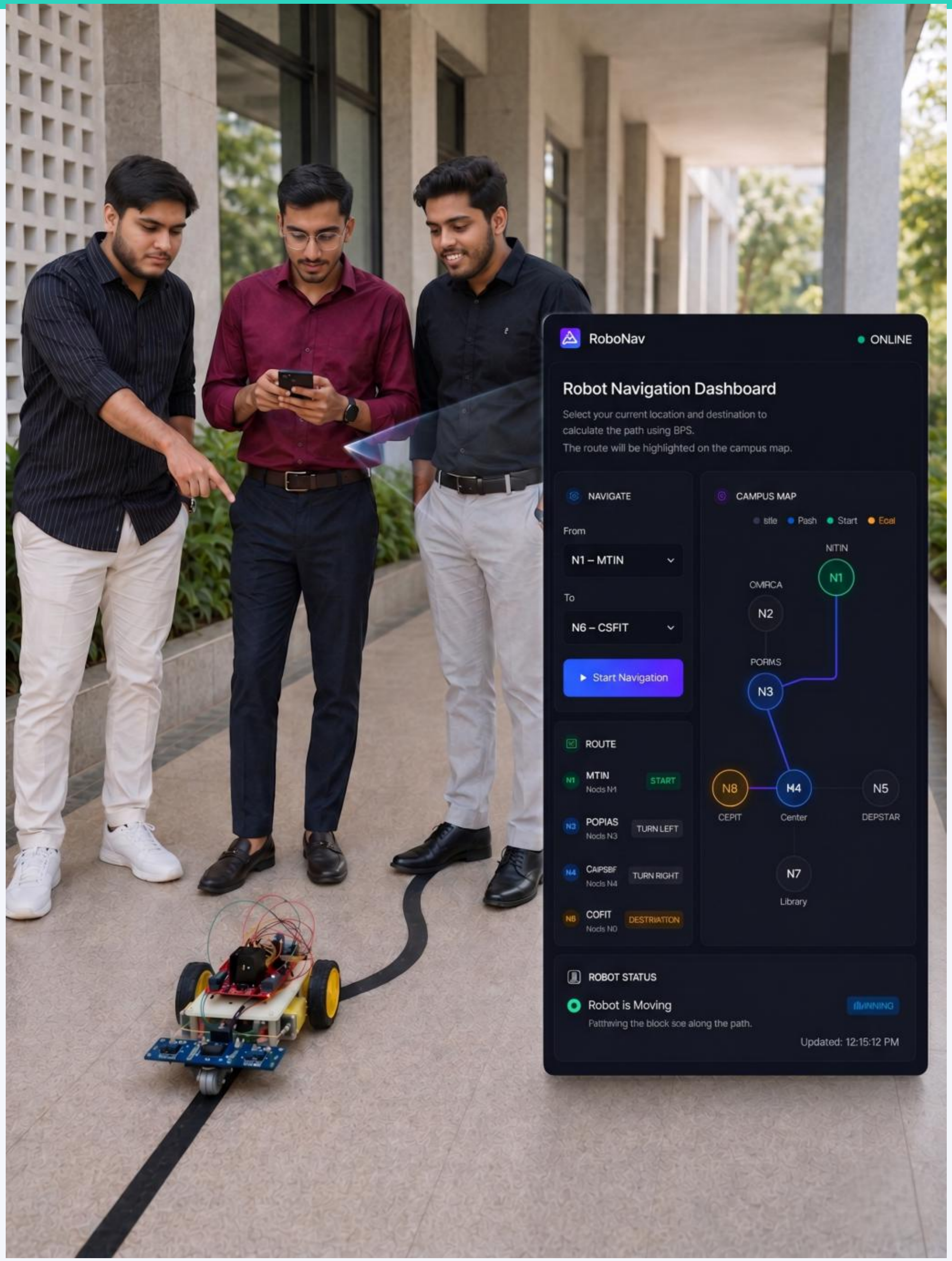
Dr. Shailesh Khant

Dean Research

CMPICA, CHARUSAT University

Introduction

Path Guiding Robot is an autonomous robot that guides visitors from one location to another on a campus by calculating the shortest path without needing any human assistance.



Problem Statement

Visitors often get lost inside large campuses.

Manual guidance requires human assistance.

Traditional robots are expensive and difficult to maintain.

Existing systems lack automation and smart navigation.

No real-time visitor tracking or analytics system exists.

Existing System — Limitations

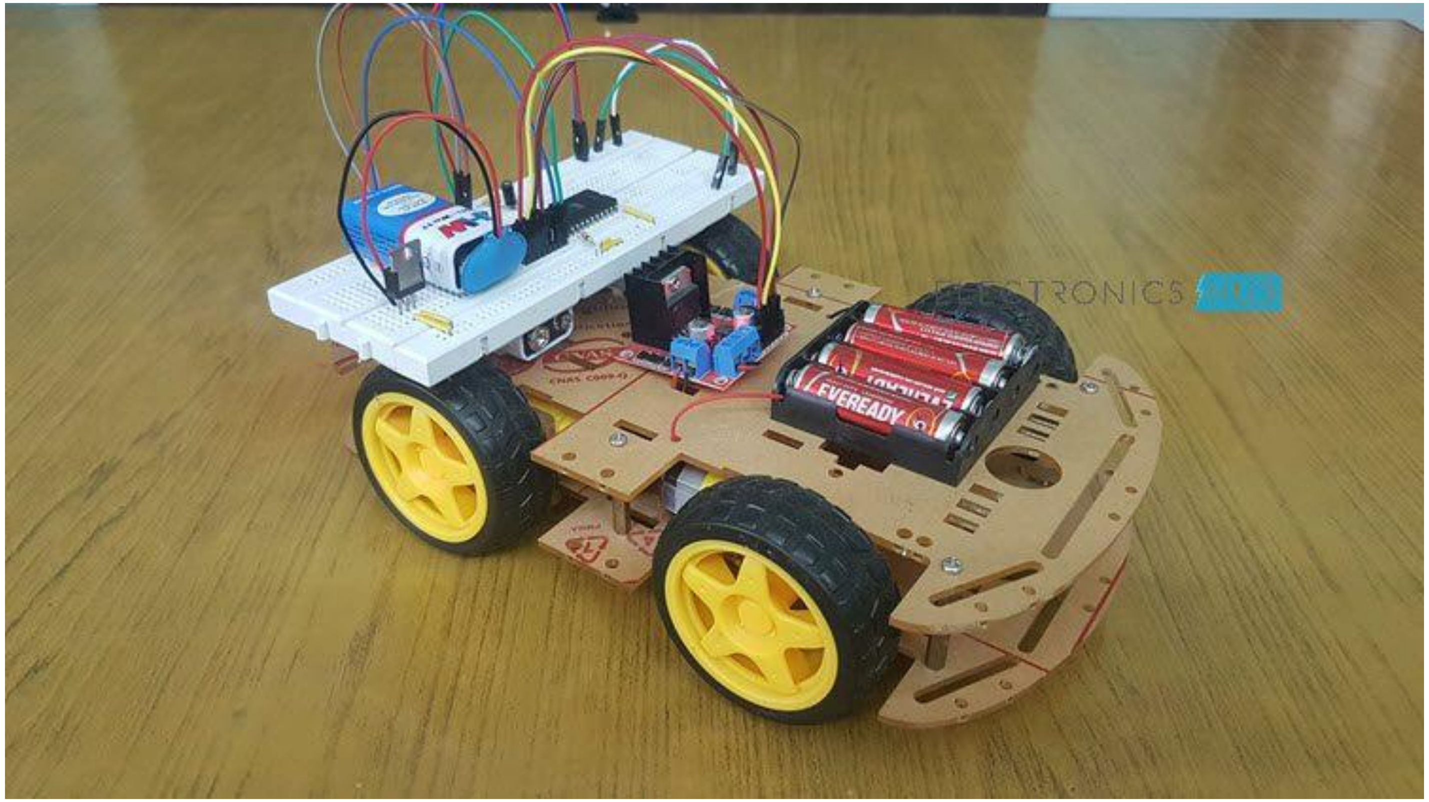
Why traditional path-following robots fall short

Traditional path guiding robots mainly use simple microcontrollers like Arduino (ATmega328) and basic IR sensors for line following.
These robots can follow only predefined paths and have limited processing and communication capabilities.
Most systems use physical buttons for control and do not support real-time monitoring or smart navigation features.
Historically, path-following robots were developed as closed-loop systems with limited external interaction.

Limitations

- Fixed destination only — robots cannot dynamically change destination based on user input.
- No web interface for remote navigation control.
- Limited wireless communication and monitoring features.
- Uses basic sensors with low navigation accuracy.
- Difficult to handle complex routes and intersections.
- No real-time robot status tracking for users.
- Expensive systems may require LiDAR, cameras, or advanced sensors.
- No analytics or visitor data monitoring system.

Existing Systems



Proposed System

An ESP32-powered, Wi-Fi connected, web-driven path guiding robot

1 User

2 RoboNav WebaApp

3 Firebase

4 ESP32 Robot

5 Destination

6 Admin Analysis

Select From → To → Enter Name → Start Navigation

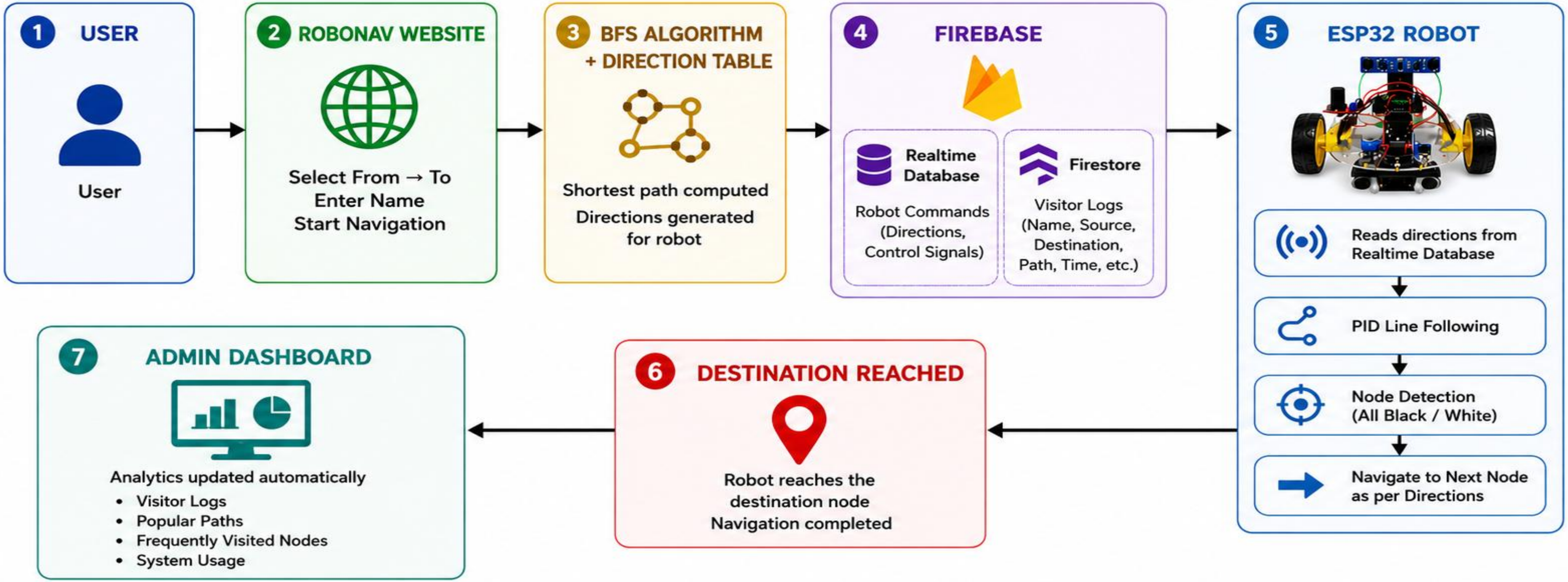
Shortest path computed, directions generated

Firebase sends commands to ESP32.

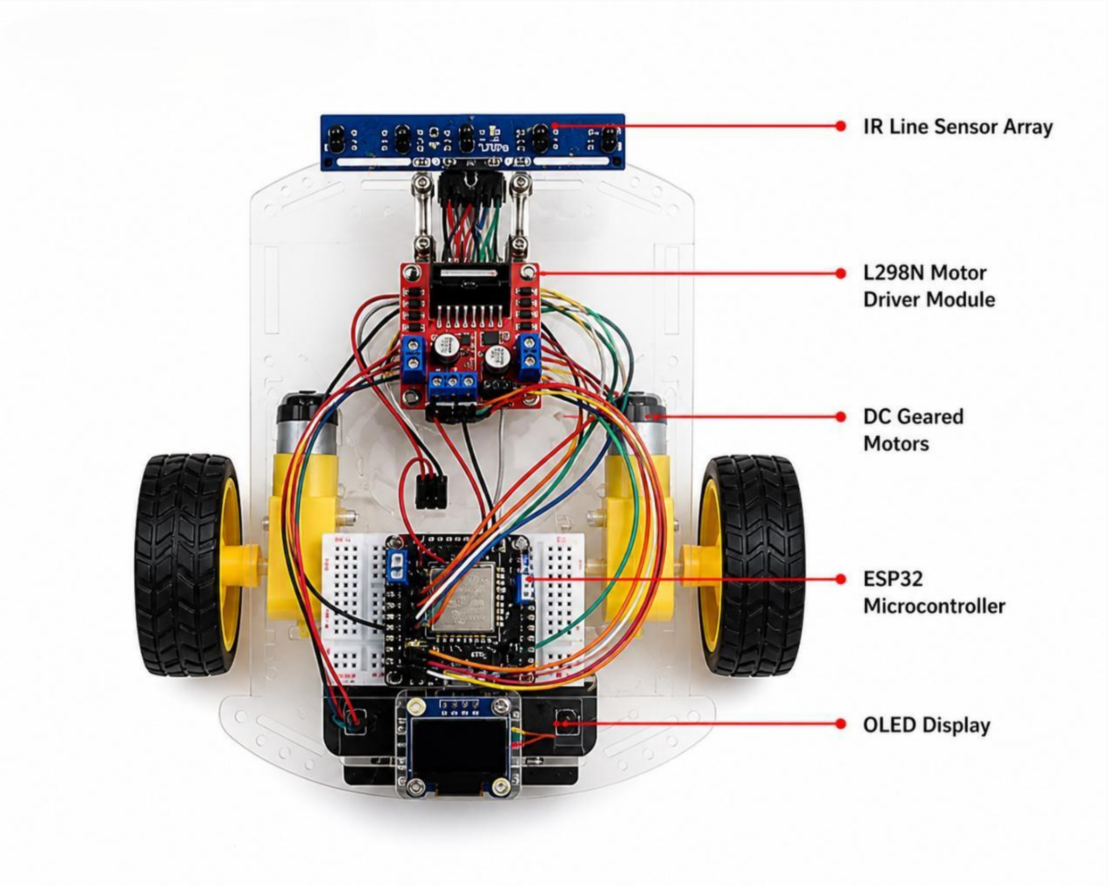
Robot follows path using PID line following.

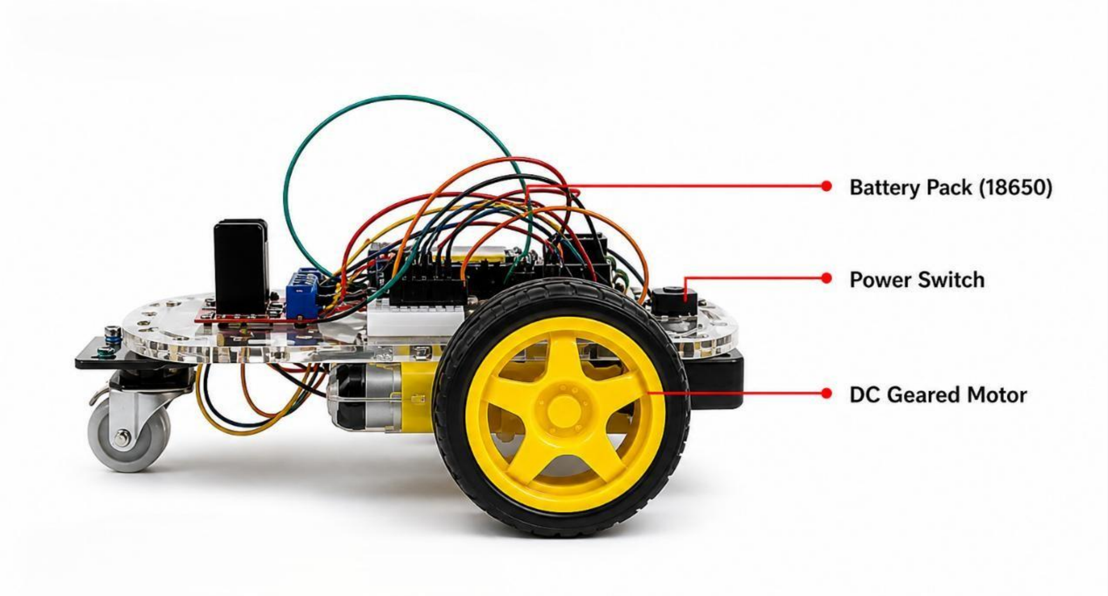
Robot status updates are shown live on website.

Proposed System



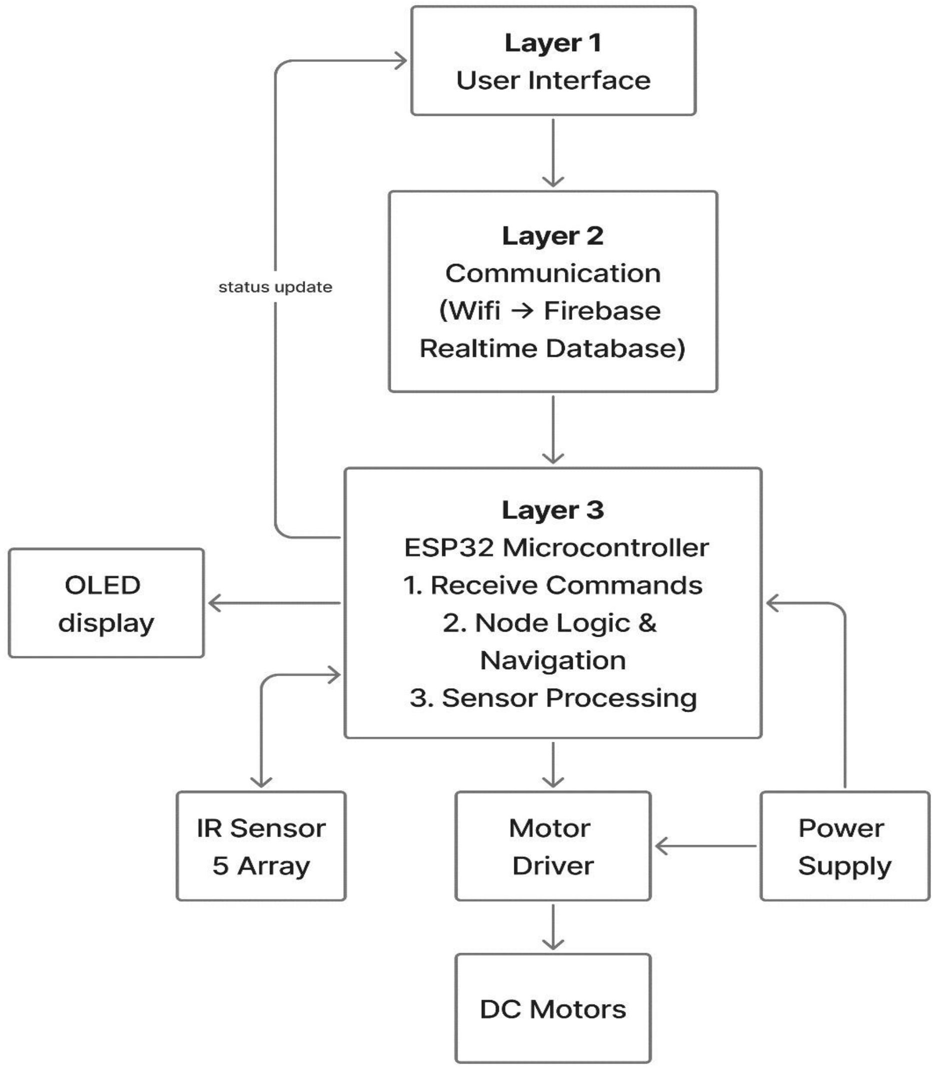
Proposed System



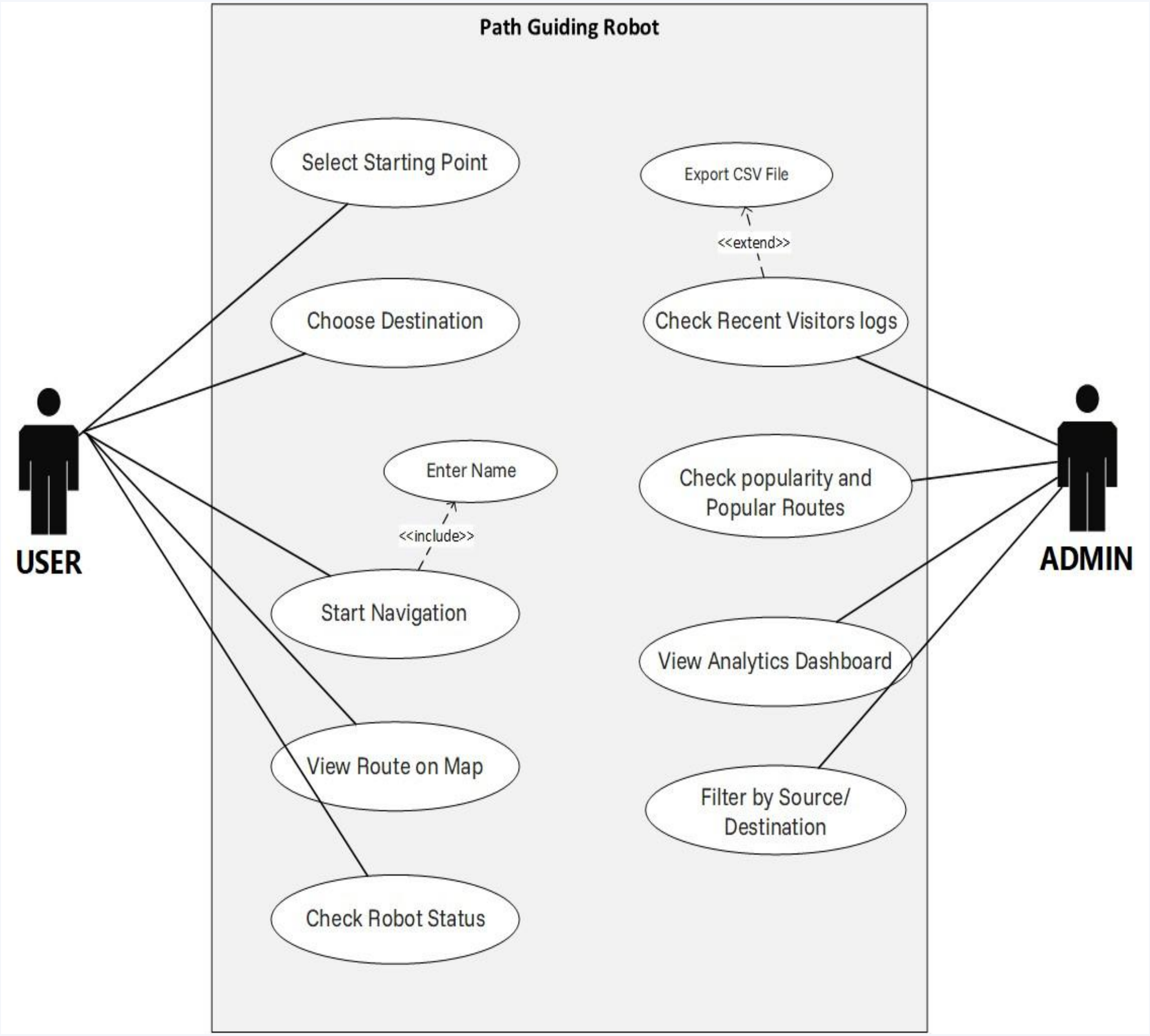


Top and Side view of Robot

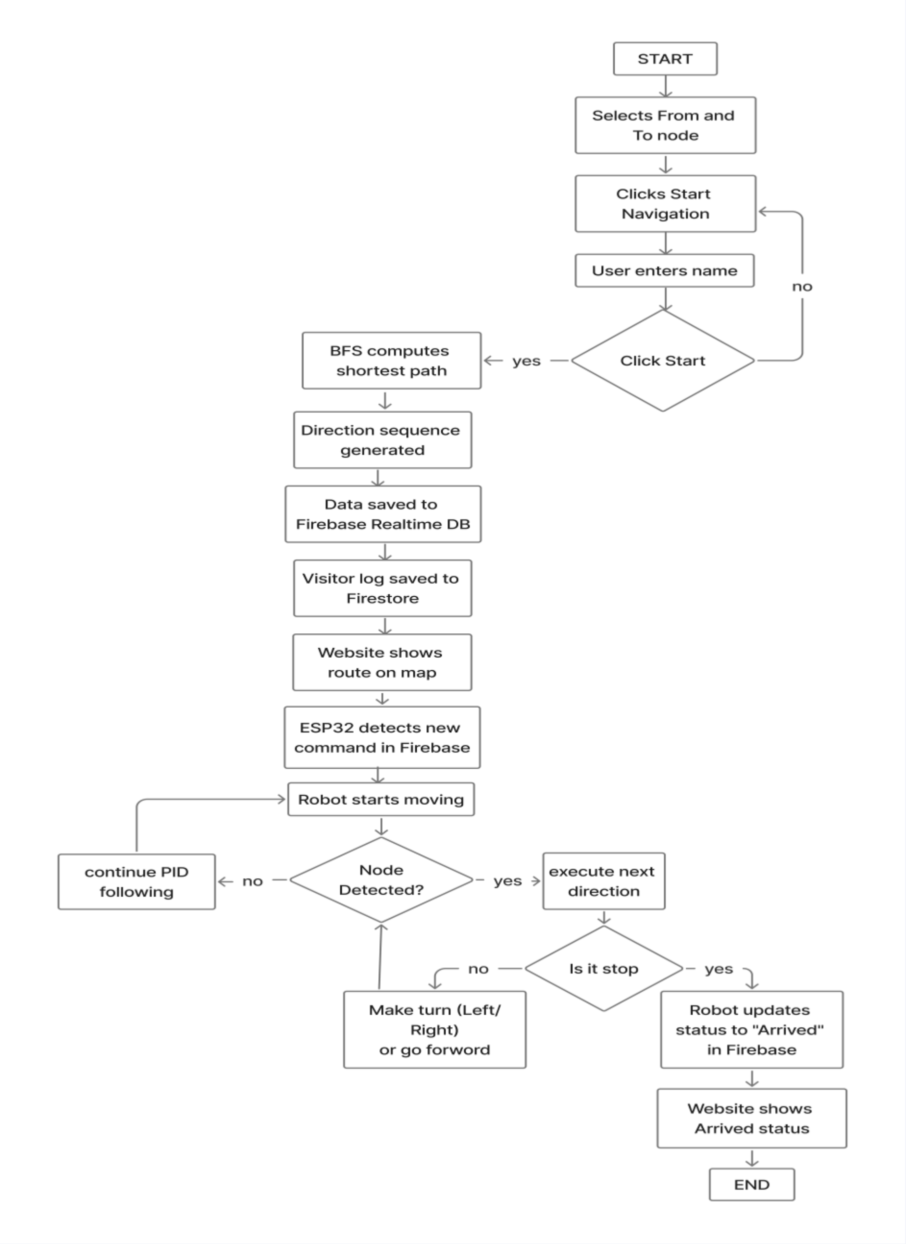
System Architecture



Usecase Diagram

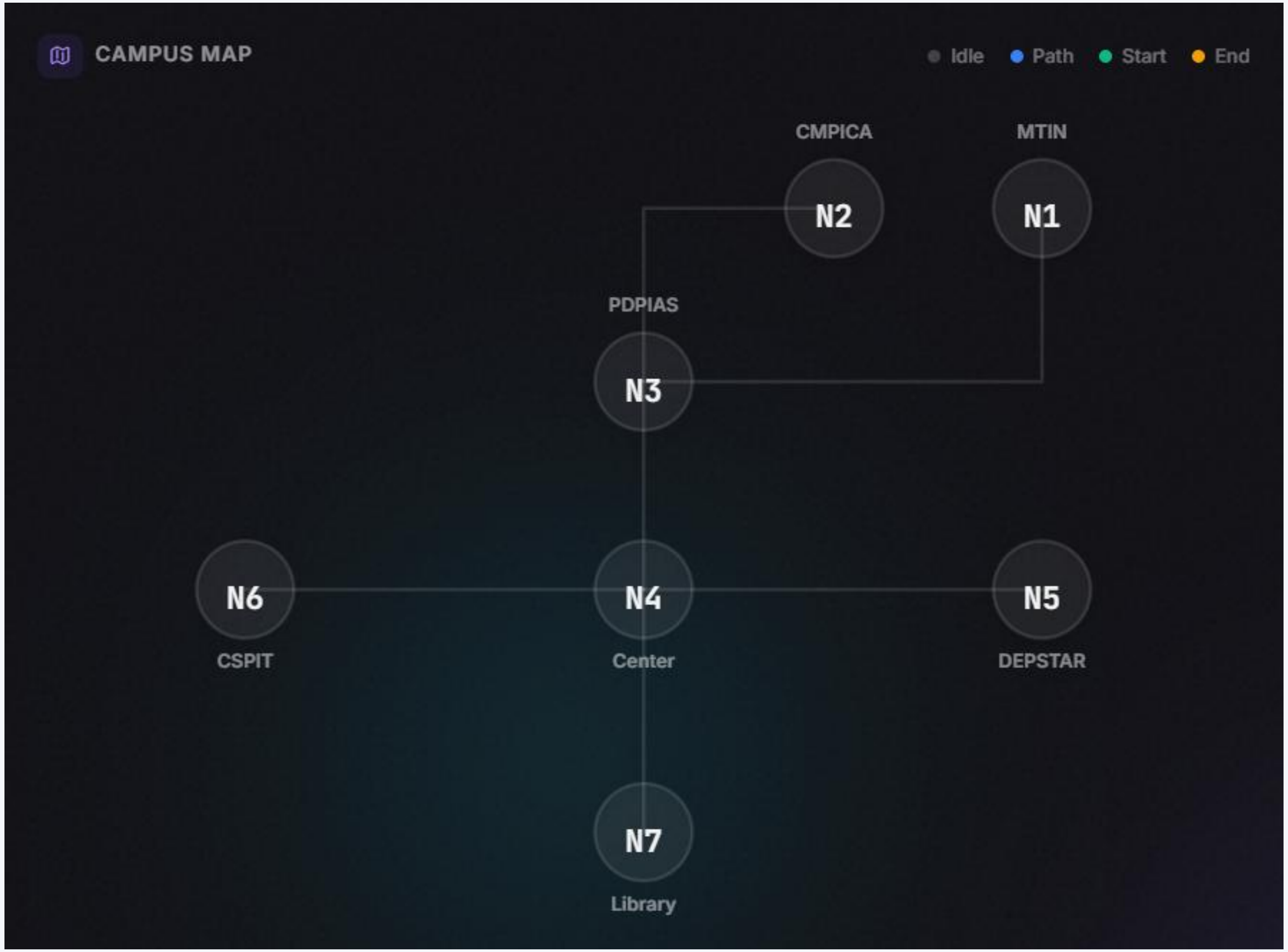


Activity diagram



Campus Map

- The campus is represented using graph nodes.
- Each node represents a building/location.
- BFS algorithm calculates shortest route between nodes.
- The selected path is highlighted on the RoboNav website.



BFS Algorithm

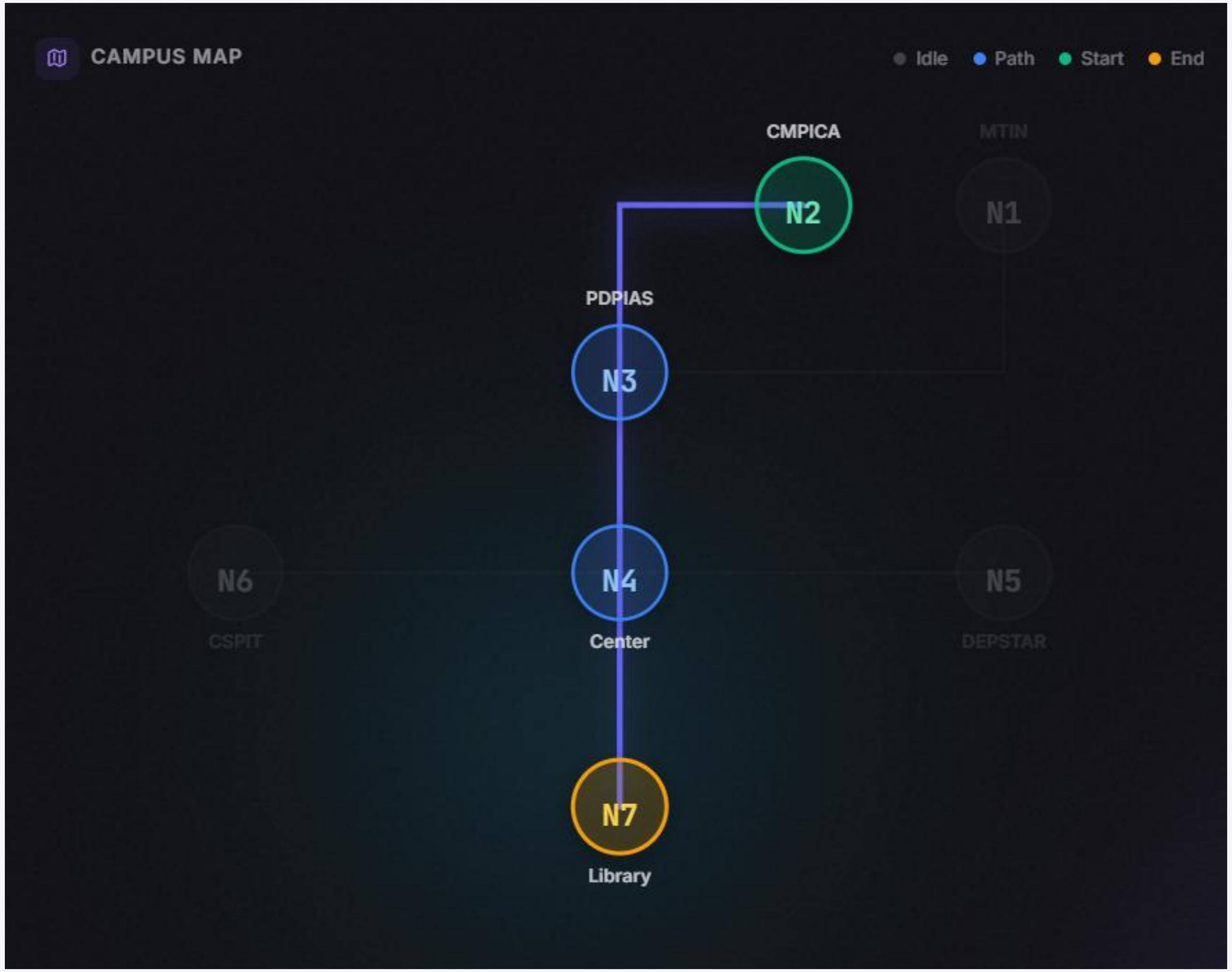
What is BFS?

Breadth-First Search explores the graph level by level from the source, guaranteeing the shortest path (in number of edges) on an unweighted graph — perfect for our campus node map.

Why BFS?

- Unweighted graph → BFS gives optimum
- Simple O(V + E) complexity
- Easy to implement in JavaScript
- Predictable & deterministic output

```
queue = [source]
while queue not empty:
   n = queue.pop(0)
   if n == dest: return path
   for nb in graph[n]: queue.push(nb)
```



Direction Table Logic

Direction Mapping

Current → Next	Action
Same heading	Forward
Turn clockwise	Right
Turn anti-clockwise	Left
Destination reached	Stop

Generated Output

```
path = [N1, N3, N4, N5]

directions = [
   "FORWARD",
   "FORWARD",
   "RIGHT",
   "STOP"
]
```

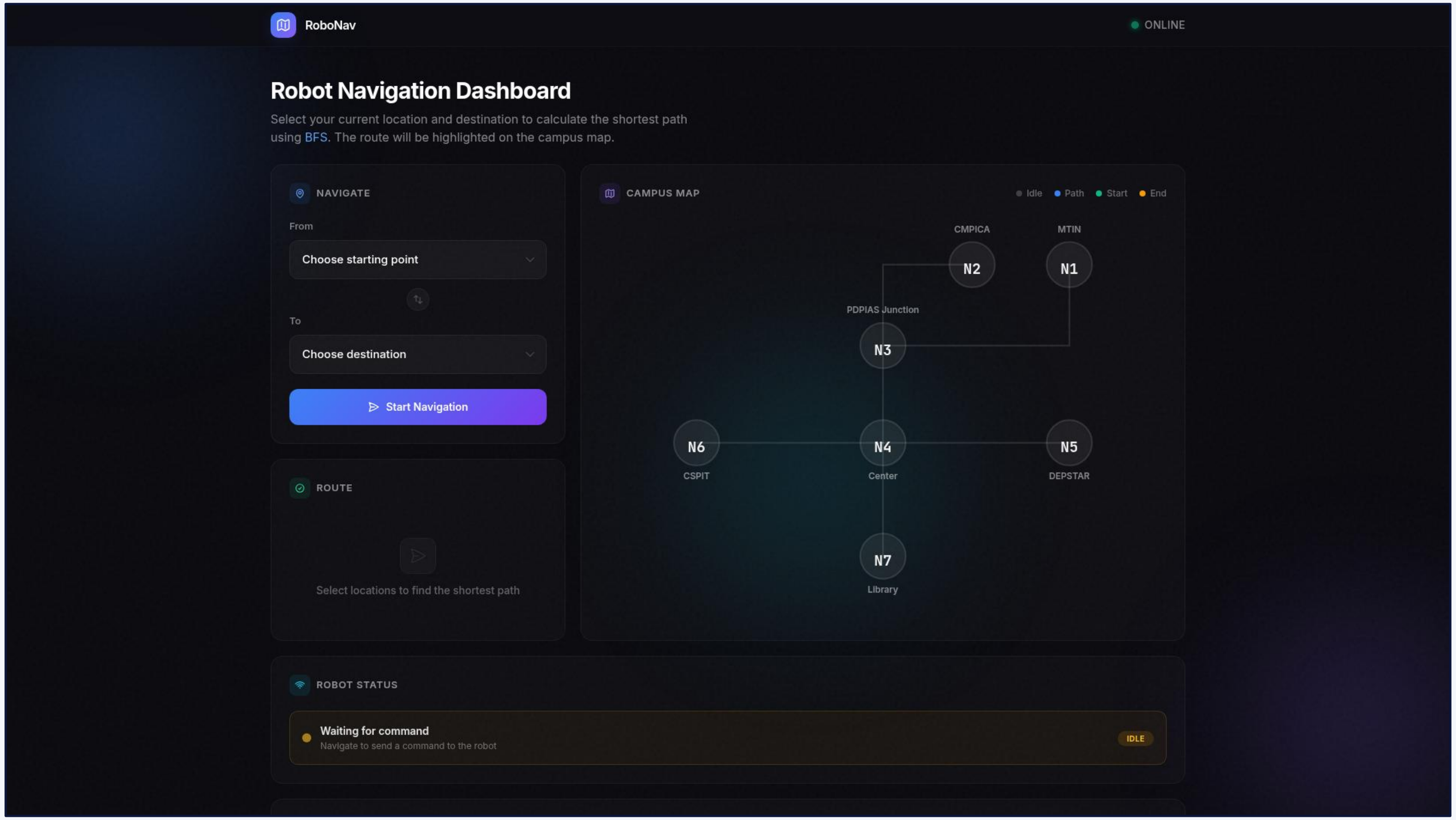
Firebase Database Structure

Real-time data fields shared between web app & robot

Field Name	Example	Description
`navigation_commandfrom`	`N1`	Node where the robot starts navigation
`navigation_command/to`	`N6`	Target node the robot must reach
`navigation_command/directions`	`Forword,Left,Right,Stop`	Ordered list of nodes from BFS result
`navigation_command/timestamp`	`1712345678`	Unix timestamp when command was issued
`navigation_command/status`	`Pending/moving`	Current robot state — Idle / Moving / Reached
`navigation_command/visitor`	`Name`	The node the robot is currently at

RoboNav Web Interface

Live web application — path-guiding-robot.vercel.app



Navigate Panel

Source & destination dropdowns + Start Navigation button

Campus Map

Visual node-edge graph; highlights path & end-points

Route Display

Shows the BFS-calculated path as an ordered node list

Robot Status

Live state from Firebase — Idle / Moving / Reached

Admin

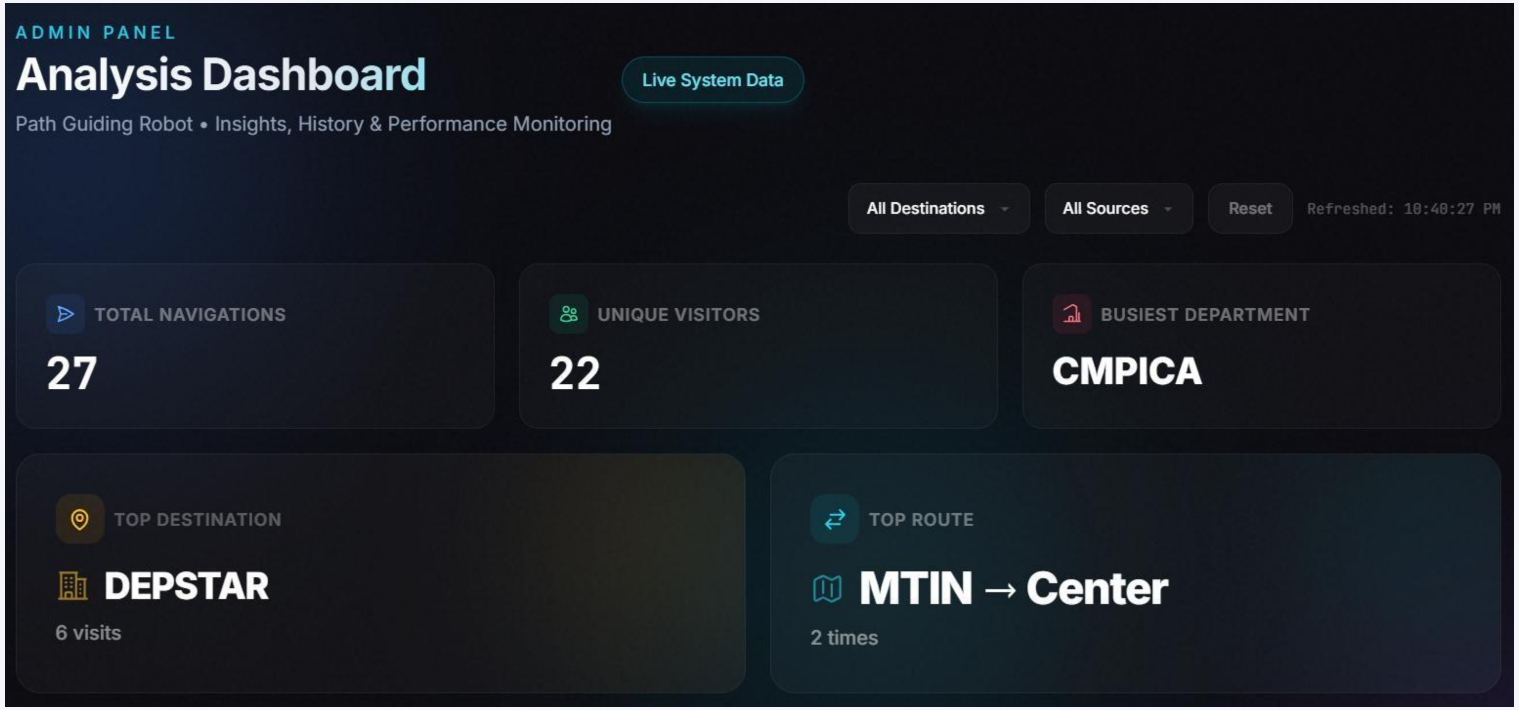
Navigation Monitoring
- Monitor total navigation requests
- Track robot activity in real time
- View current navigation status

Visitor Analytics
- Store visitor details and navigation history
- Track unique visitors
- Analyze frequently visited locations

Data Visualization
- Building popularity using Pie Chart
- Most popular routes using Bar Chart
- Recent visitor activity using Data Table

Data Management
- Filter visitor records
- Search navigation history
- Export navigation data into CSV format

Real-Time Monitoring
- Live robot status updates
- Real-time Firebase data synchronization
- Instant dashboard updates without refresh



Admin Analysis Dashboard

Hardware Components

ESP32
Wi-Fi MCU brain

L298N
Motor driver

IR Sensor
5-array line detect

OLED
I²C status display

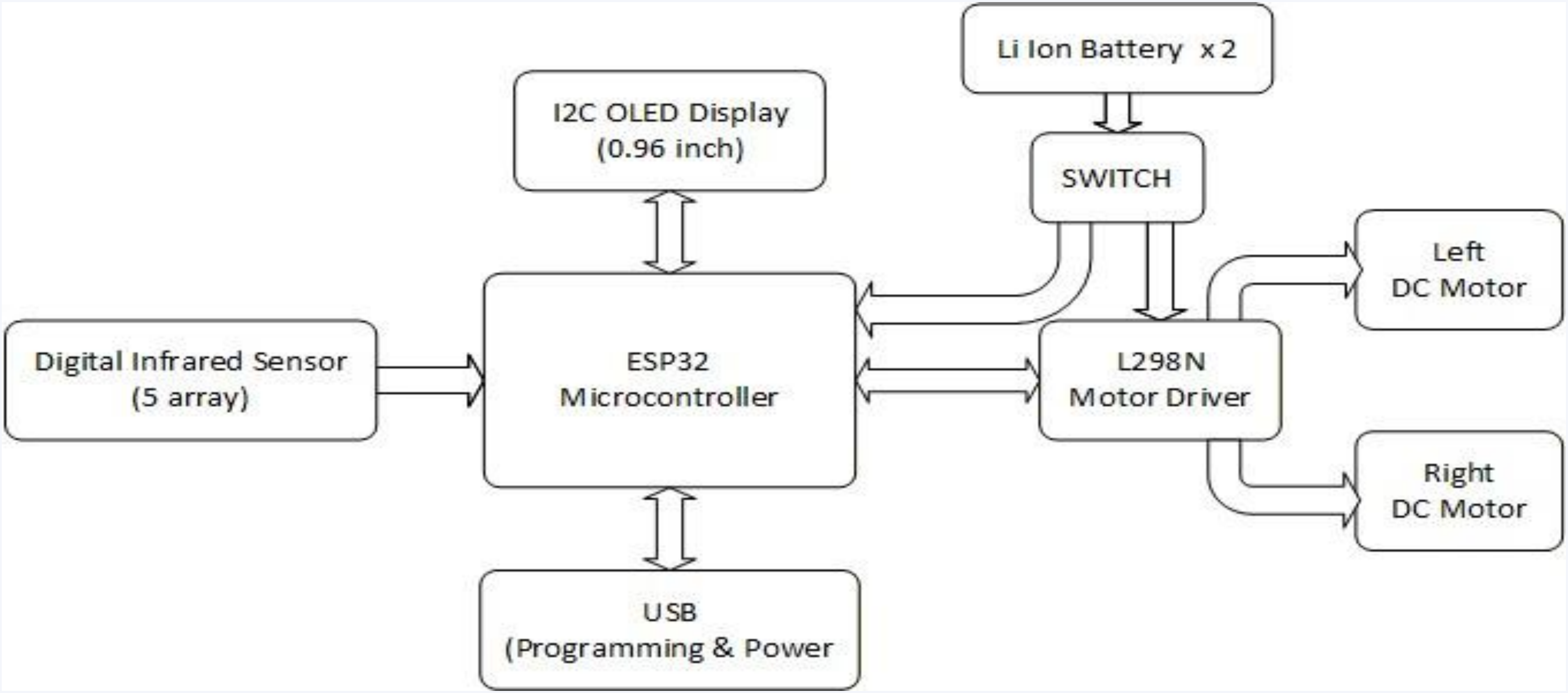
DC Motors
Drives the robot

Battery
Li-ion power

Chassis
Acrylic frame

Wheels
Castor + drive

Block Diagram



Connection Table

IR Sensor (5-array)

Sensor	ESP32
IR1	GPIO 32
IR2	GPIO 33
IR3	GPIO 34
IR4	GPIO 35
IR5	GPIO 36

L298N Motor Driver

Pin	ESP32
IN1	GPIO 25
IN2	GPIO 26
IN3	GPIO 27
IN4	GPIO 14
ENA/ENB	GPIO 12/13

OLED Display (I^2C)

Pin	ESP32
VCC	3V3
GND	GND
SDA	GPIO 21
SCL	GPIO 22
—	—

PID Line Following

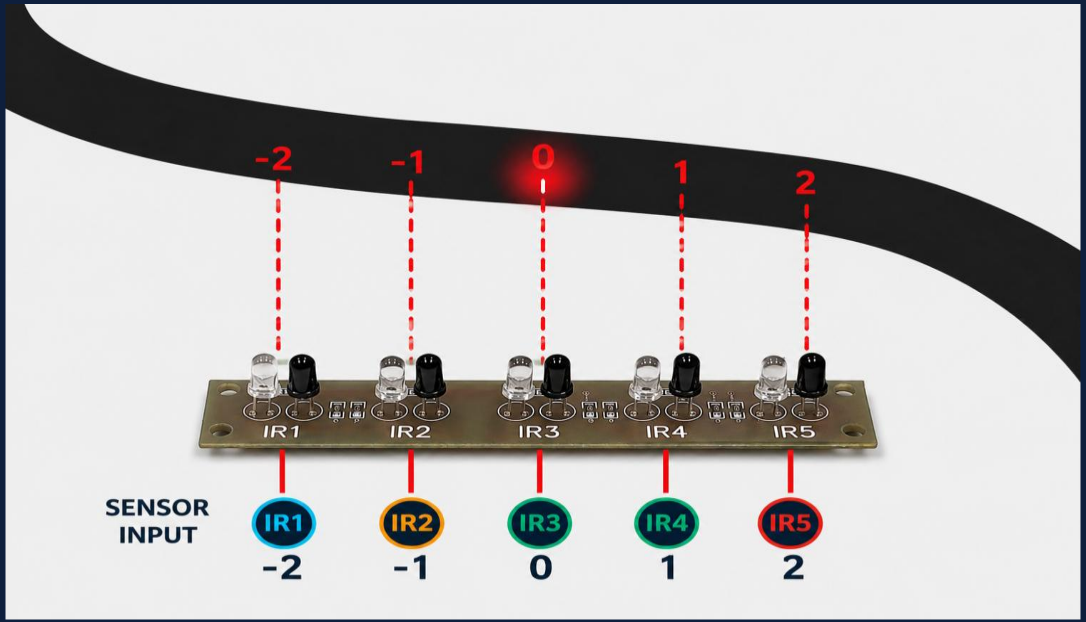
What is PID?

Proportional-Integral-Derivative controller continuously corrects the robot's position by reading sensor error and adjusting motor PWM in real time.

Why PID?

- Removes oscillation on straight lines
- Handles curves smoothly
- Adapts to surface variations
- Faster than bang-bang control

Kp	Ki	Kd
0.45	0.02	0.18



$$error = \Sigma\ (sensor_i \times weight_i)$$

$$correction = Kp \cdot e + Ki \cdot \int e + Kd \cdot de/dt$$

Node Detection Logic

Rule

All 5 sensors read BLACK
→ Node detected

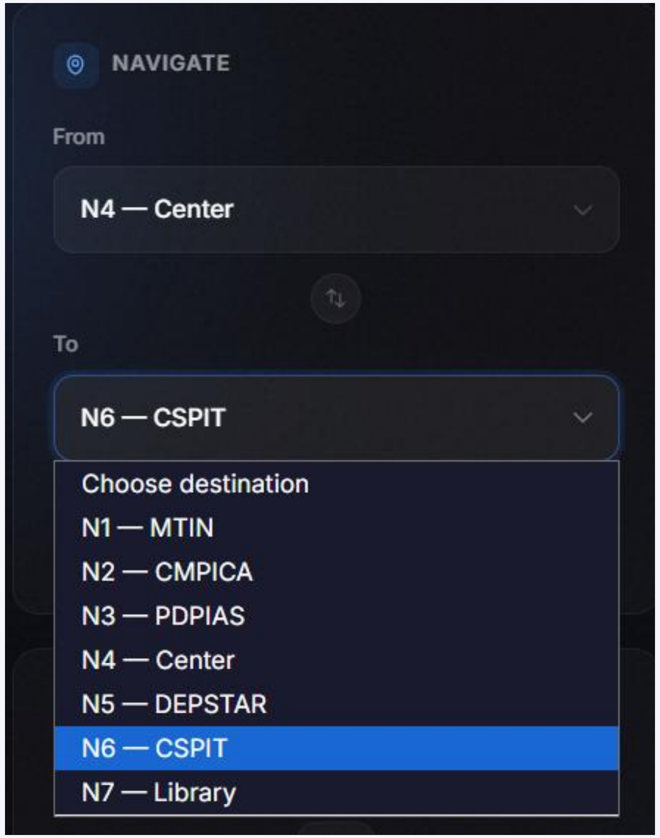
How it works

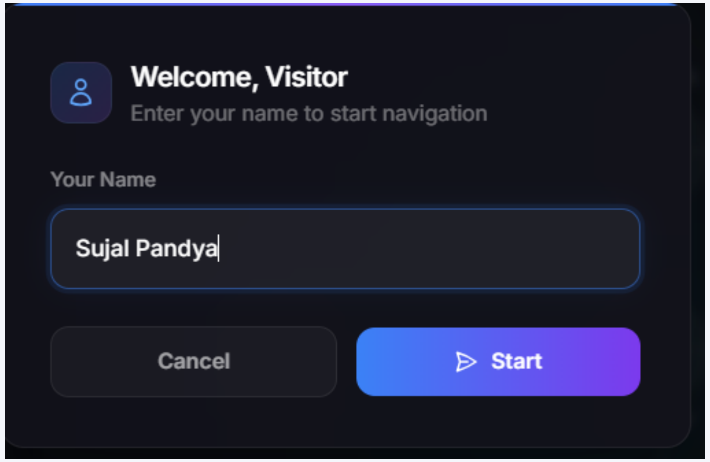
- Continuously sample IR sensors at 100Hz
- When all 5 = 1 (black), debounce 30ms
- Increment node counter
- Fetch direction[counter] from queue
- Execute Forward / Left / Right / Stop
- Update Firebase status & OLED

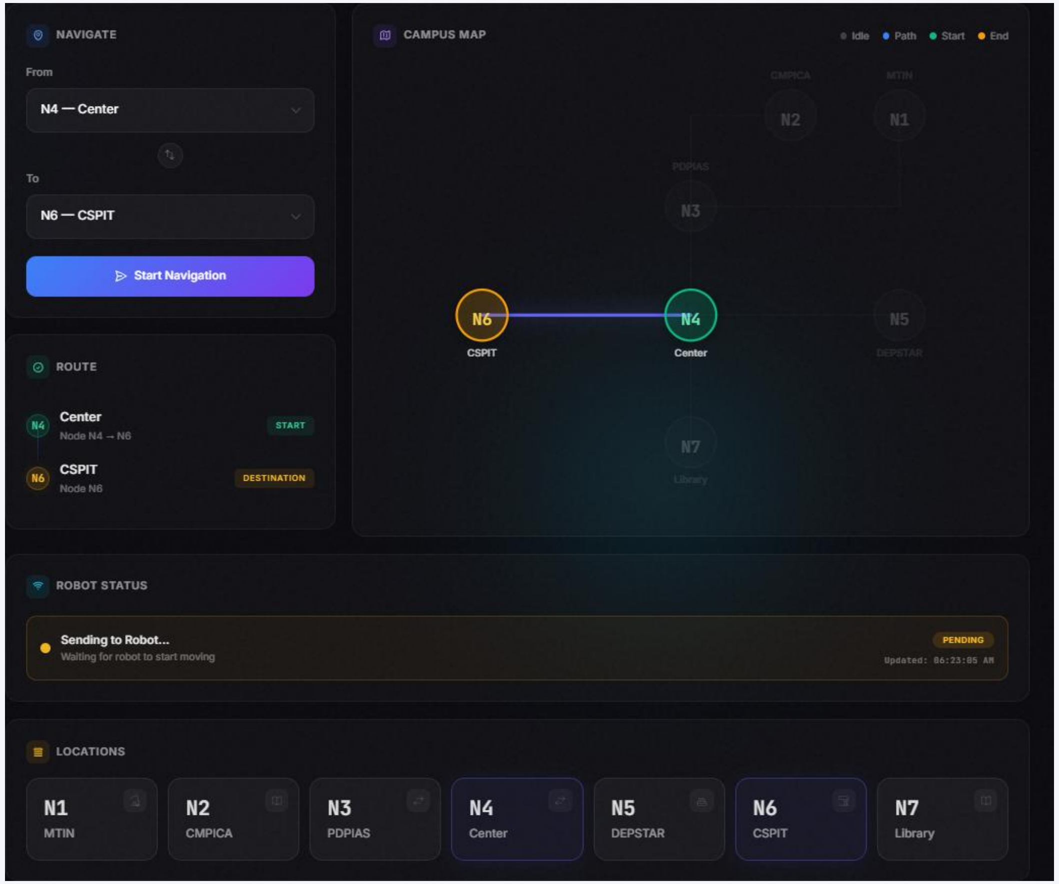
Flowchart

1. Detect Node
2. Stop Robot
3. Read Direction
4. Execute Turn
5. Continue Line Follow

Live Demo Flow

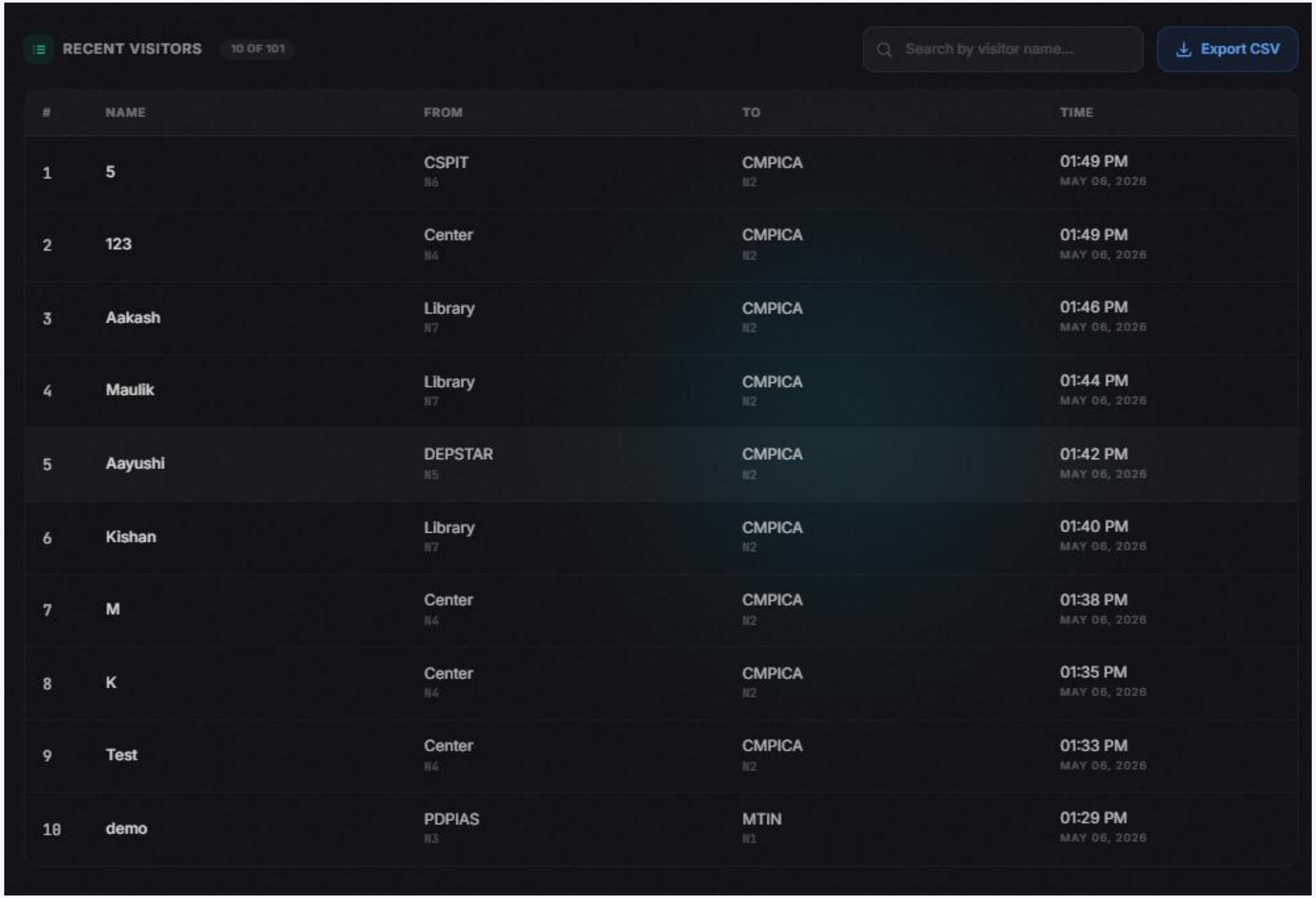






https://path-guiding-robot.vercel.app/

Visitor Analytics Flow



RECENT VISITORS 10 OF 101

Search by visitor name...

Export CSV

#	NAME	FROM	TO	TIME
1	5	CSPIT N6	CMPICA N2	01:49 PM MAY 06, 2026
2	123	Center N4	CMPICA N2	01:49 PM MAY 06, 2026
3	Aakash	Library N7	CMPICA N2	01:46 PM MAY 06, 2026
4	Maulik	Library N7	CMPICA N2	01:44 PM MAY 06, 2026
5	Aayushi	DEPSTAR N5	CMPICA N2	01:42 PM MAY 06, 2026
6	Kishan	Library N7	CMPICA N2	01:40 PM MAY 06, 2026
7	M	Center N4	CMPICA N2	01:38 PM MAY 06, 2026
8	K	Center N4	CMPICA N2	01:35 PM MAY 06, 2026
9	Test	Center N4	CMPICA N2	01:33 PM MAY 06, 2026
10	demo	PDPIAS N3	MTIN N1	01:29 PM MAY 06, 2026

Test Cases

TC ID	Component	Description	Expected Result	Status
TC-01	ESP32	Verify Wi-Fi connection	ESP32 connects and shows IP address	PASS
TC-02	Firebase	Verify navigation data reading	ESP32 reads source, destination, and directions	PASS
TC-03	Firebase	Verify status update to database	Firebase shows Online status	PASS
TC-04	L298N Motor Driver	Verify motor movement	Motors move correctly in all directions	PASS
TC-05	IR Sensor	Verify line detection	Sensors detect black and white surfaces correctly	PASS
TC-06	PID Logic	Verify straight line following	Robot follows line smoothly without deviation	PASS
TC-07	PID Logic	Verify curve navigation	Robot follows curved path accurately	PASS
TC-08	IR Sensor	Verify node detection	Robot detects intersection/node successfully	PASS
TC-09	Motors + IR	Verify turning at node	Robot performs correct 90° turn	PASS
TC-10	Firestore	Verify visitor log storage	Visitor data saved successfully	PASS
TC-11	ESP32 + IR	Verify safety stop	Robot stops when line is lost	PASS
TC-12	OLED Display	Verify display information	OLED shows Wi-Fi, IP, and direction status	PASS
TC-13	Wi-Fi Failure	Verify reconnection handling	Robot reconnects after Wi-Fi restoration	PASS
TC-14	Full System	Verify complete navigation flow	Robot reaches destination successfully	PASS

Advantages

Low Cost

Built with affordable components

Real-time

Sub-second cloud communication

Cloud Integrated

Firebase RTDB + Firestore

Autonomous

BFS + PID self-navigation

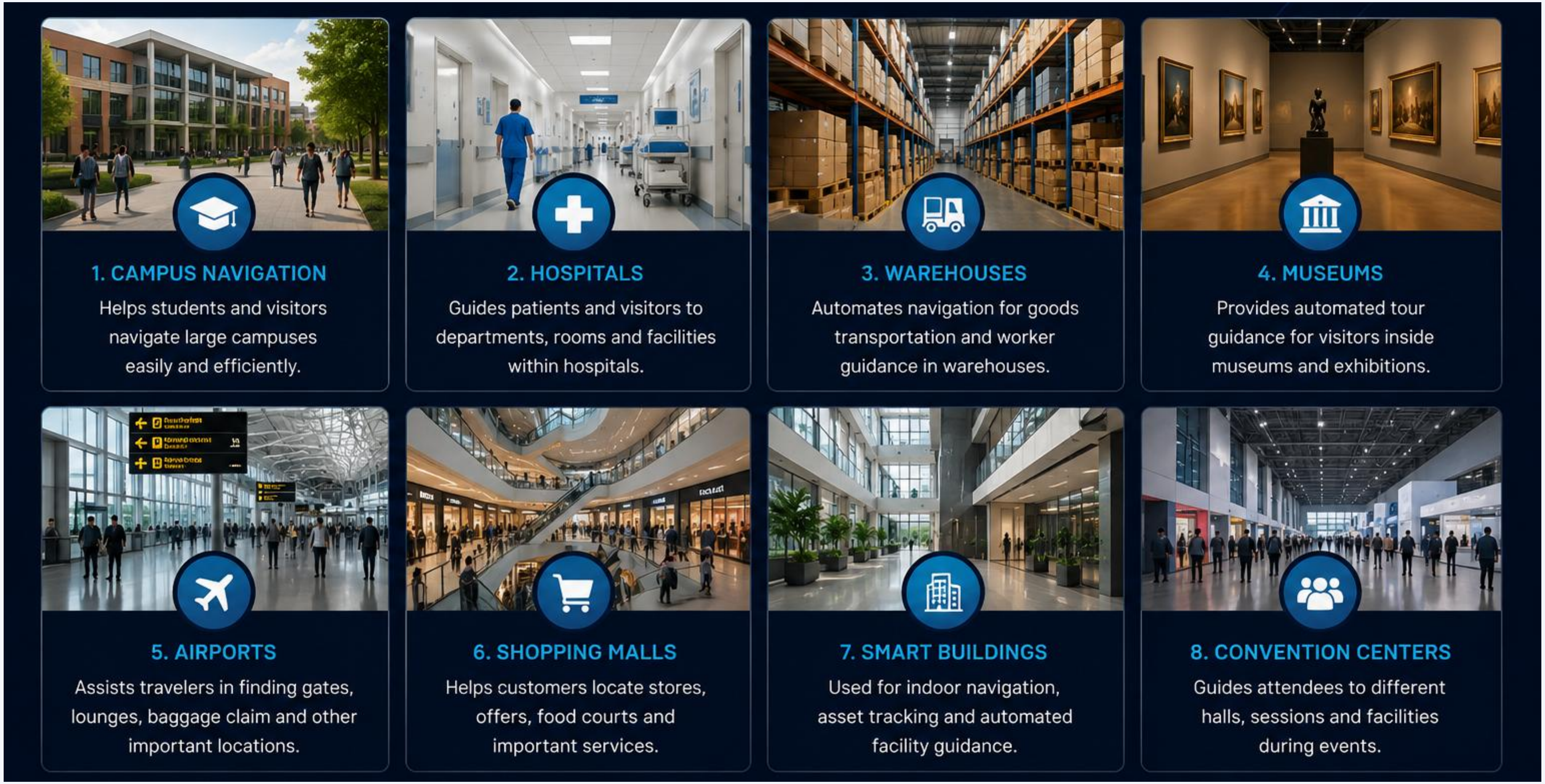
Analytics

Visitor data & insights

Modular HW

Easy to extend or repair

Application



Future Enhancements

Where the project goes next

▣ Obstacle Detection

Add HC-SR04 ultrasonic or IR proximity sensor — robot pauses if a person/object blocks the path and resumes once clear.

◉ Live Location Tracking

Animate the robot's exact position on the campus map in real time using node-by-node Firebase updates.

◐ Dynamic Map creation

Modify map as require without changing internal logic

▤ Mobile Application

Dedicated Android/iOS app using device GPS to auto-detect the user's nearest campus node.

✸ Solar / Wireless Charging

Solar panel or wireless charging pad at the home base keeps the robot powered all day.

Conclusion

Achievement

Successfully delivered a fully working IoT-based Path Guiding Robot — combining a web interface, Firebase cloud, BFS shortest-path planning, ESP32 firmware and PID line-following — into one integrated, demonstrable system.

BFS

Shortest path on the campus graph

PID

Smooth & accurate line following

Firebase

Real-time web ⟷ robot bridge

SCALABLE

Built for a smart-campus future — easily extendable to hospitals, airports & multi-robot fleets.

A Novel Robotics Design for Campus Path Guiding and Live Tracking System for Visitors in Educational Institutes of Gujarat

Maulik Pithiya
Dept. of Computer Science and Applications
Charusat, Changa
23bsit119@charusat.edu.in

Abhishek Parmar
Dept. of Computer Science and Applications
Charusat, Changa
23bsit060@charusat.edu.in

Sujal Pandya
Dept. of Computer Science and Applications
Charusat, Changa
23bsit055@charusat.edu.in

Shailesh Khant
Faculty of Computer Science and Applications
Charusat, Changa
shaileshkhant.mca@charusa..ac.in

***Abstract*— This Research is a modified version of path guiding because traditional path-following robots rely on limited microcontroller processing power lack wireless communication capabilities and they were working on fixed non junctional path. These existing systems fail to prove real-time status monitoring and also in capable to run BFS algorithm. To address this limitation, this project proposes an IOT-based autonomous Path Guiding robot system for campus navigation. The solution comprises a Web-based interface(User and analysis side) integrated with Breadth-First search(BFS) algorithm to compute shortest path and generate turn-by-turn navigation .The hardware implementation include an ESP32 microcontroller as the central processing unit linked to an motor driver, Array IR sensor module and OLED display. The bidirectional data communication and remote monitoring are achieved using Firebase Real-time Database and Firestore. For maintaining the speed, turns, oscillation, line following we have used PID control algorithm. System testing result demonstrate successful end-to-end routing, visitor data logging and a low-cost deployment. The Real-world implementation of this system can be in university campus, Warehouse navigation and in restaurant for taking and serving order.**

***Keywords*— IoT, Embedded system, Line Tracking, Analysis**

I. Introduction

The system is the merge of two type of component Software and Hardware. The Software side consist of web-interface called Robonav which allow user to select a starting point (Source node) and an ending point (Destination node) from a fixed campus map. Upon clicking Start Navigation user have to enter name which is logged into Firestore which can be used for analytics purpose. Then Breadth-First Search(BFS) algorithm calculate the shortest path between selected nodes and generate a step-by-step direction sequence for the navigation of the robot.

On hardware side, the ESP32 microcontroller connect to Wi-Fi and receive data from Firebase Real-time Database for navigation commands. Once a command is received, the robot begins execution. A 5-array IR sensor module detect the black line on the ground and provides a digital reading to the ESP32. A PID control algorithm is implemented which processes these sensors reading to calculate real-time motor correction, ensuring smooth and accurate line following. The node is detected when all five sensors detect black line at the same time and the form there the robot execute next direction based on the path sequence. The speed and direction of motor are controlled with the help of L298N motor driver which receives PWM signal from ESP32. An OLED display is connected which provide a real-time status including Wi-Fi connection status, IP address, current direction and on other details.

All connections between component are done through a breadboard this make platform modular and can be easily modified. To provide real-time visibility of robot's operational state, the ESP32 microcontroller continuously writes its current status back to the Firebase Database throughout the navigation process. This bidirectional communication process ensures that web interface (RoboNav) always reflect the latest state of robot without any manual refresh. ESP32 connection status is reflected on website (Online/Offline). There is a live Status feedback which allow user to monitor the robots progress from web interface during navigation, this makes the system fully transparent and interactive from both ends.

The web interface also allow user to analyze the overall navigation data and based on that user can know about different popular roots, filter data and export all navigation data in a csv file.

In the past, path following robots are not advance and use old technology. There are broad size microcontrollers, heavy drivers and components are used which is hard to manage also does not have much processing power without wireless connectivity. These robots relied on one or two IR sensor, which only follow very basic paths and struggle with nodes and intersections.

Limitations

- Most path following robots are used to program for fixed destination
- No web interface only physical buttons are mounted.
- Multiple systems needed for full functionality
- No real time status monitoring and tracking.
- Not collect any type of data which can be useful for analysis.

Fig. 1 shows a block diagram illustrates how an autonomous navigation system works with the ESP32. The ESP32 microcontroller acts as a central processing unit (CPU) within this system this is where all the information that allows the robot to navigate successfully comes together.

The ESP32 receives information from 5 infrared (IR) sensors that can detect lines on the ground or floor. The CPU uses a Proportional-Integral-Derivative (PID) control algorithm to process these signals for accurate and efficient control of the robot's movements. In addition, the CPU will use a L298N motor driver module to power 2 motors that allow the robot to navigate among straight lines, curves, and turns.

Performance commands sent wirelessly via web interface (via Firebase) will be interpreted by the ESP32-Central Processing Unit; thus, it will send commands to the driver modules to control the motion of the motors (forward, backward, or turning).

The status of the robot (moving toward a node or moving forward) will be available to users through the OLED screen (using I2C communication). Power is supplied via 2 Lithium-Ion batteries via a switch module.

The robot uses PID for line-following using a 5-array IR sensor to accurately follow straight paths, gradual curves, and 90-degree junctions, while also detecting nodes and executing directions as required. Additionally, the system provides real-time status updates on the web interface and also includes an admin dashboard for visitor logging and campus analytics.

Fig. 2 consist of activity diagram for working of path guiding robot. Where the process starts with user open the web interface then chooses their current node and selects destination node. The user then initiates navigation by providing their name. The system uses Breadth-First Search (BFS) algorithm which help to compute the shortest route and then it generates a data of directions. The path and direction of the data are saved to the Firebase Database. Then the visitor log is saved to Cloud Firestore.

The ESP32 continuously receives command from firebase and based on that motor are activated. The robot follows physical path by using a 5-array IR sensor which is regulated by a PID control algorithm. When these sensors detect an intersection between node, the robot executes next direction from the sequence. When it arrives at destination it shows status "Arrived" in web interface.

It is important to note that the system is limited to navigation within a predefined campus map and requires a stable Wi-Fi connection for operation. The current implementation uses a physical track represents the campus layout of CHARUSAT, consisting of a black line on a white surface, and serves as a proof-of-concept for indoor campus navigation assistance.

Fig.2 Activity Diagram

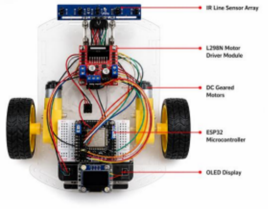


Fig.3 Path Guiding Robot Hardware

Fig. 3 shows the hardware setup of path guiding robot system. This includes key components such as the ESP32 microcontroller, 5-array IR sensor for path detection, L298N motor driver module which is used for motor control, DC motors for movement of robot and an OLED display which shows status. Other than that, it includes power and drive components which includes 18650 battery pack for power supply, a switch for system control, and DC geared motors for movement of wheels. Overall, all these components all work together to start real-time navigation and path following of the robot.

Fig. 4 includes a IOT Architecture Diagram In which it consists of three layers. Layer one is user interface where the data which was entered by end user is taken. Layer two is for communication between hardware and software which is done by Wi-Fi and firebase. Layer three is for CPU of system which is ESP32 which receive command, implement node logic and navigation, sensor processing also done by it. ESP32 is connected to OLED display to show status,5-Array IR sensor for receiving line following input, Motor Driver Which is connected to DC motor which lead robot to move and power supply helps to supply power to overall system.

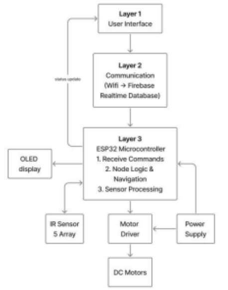


Fig.4 Architecture Diagram

V. RESULT AND DISCUSSIONS

A Path Operating Robot with Smart Campus Analytics will offer a cost effective and efficient alternative for covering an indoor autonomous navigable space on a pre-defined campus, which also incorporates the usage of a web interface, BFS path planning algorithm, Firebase cloud communication, PID controlled line following and real time node location detection as one single platform.

With this robot the end-user will have the ability to select the starting point and end point of the robot's route; after that point the robot will operate completely autonomously without any human intervention. This system will also record and retain all activity to Firebase to provide an administrative dashboard that can give a wealth of information regarding how visitors move about the campus as well as how often the campus is used. As a result of seamless interaction between all hardware components and all software components and the ability to provide real time communication between the hardware and software, the system will provide accurate navigation and optimal overall performance. The system uses inexpensive hardware that may be easily installed and many of the available tools are open-source; thus, resulting in a very scalable and viable option for smart campus solutions.

Fig 5 shows that there is a home screen consists of from/to dropdown menus for selecting starting node and ending node. Then a Start Navigation button which lead to enter name after that navigation will start. Web Interface also consists of campus map graph which show the overall sorted path in graphical manner and also a robot status panel which show status of robot (Pending/Running/Reached).

Fig. 6 figure represent screen for admin dashboard which can be used for monitoring and analyzing system usage. It displays data in statistical manner such as navigation done by users and visual insights which including building popularity (pie chart) and most frequent used routes by user. User can also export all visitor data in csv file and also filter data.

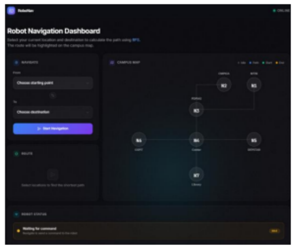


Fig.5 Home Page

Adding ultrasonic sensor-based obstacle detection is important in order to improve the system's performance in a real environment. The system should be designed to support dynamic path recalculation for blocked paths and to allow adaptive navigation so that users can continue their course through an environment. Analytics modules will also be developed to implement server-side solutions for analytics that include traffic patterns, predicted outcomes, and other forms of advanced analytics (e.g., Firebase Cloud Functions).

Additional plans for enhancing the system may include developing multi-robot coordination for the concurrent operation of multiple users; a voice guidance solution to help to improve the user interface; and a new custom PCB for

THANK YOU

■ TRY THE LIVE WEB APP

path-guiding-robot.vercel.app

Live navigation dashboard — try selecting a route

■ PROJECT TEAM

Maulik Pithiya • Sujal Pandya • Abhishek Parmar

CMPICA, CHARUSAT — April 2026